Del demo al repo real — qué se implementó y prompts en orden

- Demo completa: `C:\Users\Andrey\Downloads\ProyectoWeb-main\ProyectoWeb-main`
- Repo real: `C:\Users\Andrey\source\repos\Proyecto` (rama `feat/asset-maintenance-database`, último commit d842cf9)

Los prompts se ejecutan **hoy, uno detrás del otro y en este orden**: cada uno depende del anterior. Después de cada prompt: compilar, revisar el diff, probar en Scalar lo que se tocó y hacer el commit sugerido (sin línea de co-autoría). La columna "Quién" reparte los commits de forma pareja; si uno de los dos aplica todos, igual queda bien.

Cada prompt le dice a Claude Code que **lea la demo como referencia**: así copia la misma solución en lugar de inventar otra. Si Austin no tiene la demo en su computadora, pásale el zip.

---

1. Qué se implementó en la demo

#	Cambio	Rubros que cubre
1	Excepciones de negocio propias (`ExcepcionNegocio` → `ExcepcionSolicitudInvalida` 400, `ExcepcionNoEncontrado` 404, `ExcepcionConflicto` 409). Los servicios ya no devuelven `Resultado.Fallo`: lanzan la excepción.	A5
2	`ManejadorGlobalExcepciones` (`IExceptionHandler`): convierte las excepciones de negocio en 400/404/409 y cualquier otra en 500, siempre con `Resultado`. Registra Warning (negocio) o Error (inesperado).	A4, A5
3	`ResultadoBase` con `timestamp` y `traceId` (solo en errores); la respuesta de validación 400 también lleva `traceId`.	A6
4	Serilog (consola + archivo diario en `Logs/`, no en el entorno Testing), `UseSerilogRequestLogging`, niveles en `appsettings.json`.	A4
5	Todos los controladores: `ILogger<T>` con Information al iniciar y al terminar, `/// <summary>` y `[ProducesResponseType]` con el tipo de cada respuesta. `GenerateDocumentationFile` en el csproj para que Scalar muestre los resúmenes.	A3, A4, A8
6	Activos: filtros `?categoryId=&status=` validados con DataAnnotations en los parámetros (400 si el estado no existe); sin `status`, el listado oculta los dados de baja, como el `Status == true` de clase; `categoryId`, `ageInMonths` y `maintenanceOrderCount` en el listado. El detalle trae además `category` (id, nombre y umbral), `hoursSinceLastMaintenance`, `preventiveMaintenanceDue` y `maintenanceOrders` (código, fecha, tipo, prioridad y estado).	C5, C8, C9, C14, C16
7	`PATCH /api/Asset/{id}/status` acepta también `Decommissioned` y responde 409 si el activo tiene una orden abierta. Como en el patrón de clase (`GetForUpdateAsync` excluye los eliminados), un activo dado de baja responde 404 en PUT, PATCH, DELETE e imagen; el GET por id sí lo muestra para conservar su historial.	C11, C12, C19

#	Cambio	Rubros que cubre
8	Categorías: `assetCount` en el listado y en el detalle, y `assets` (código, nombre y estado) en el detalle. Se corrigió el nombre JSON `codePrefix`.	D6, D9
9	Órdenes: 404 si el activo no existe (lo pide la rúbrica); 400 si el tipo o el proveedor no existen (patrón de clase: un Id inexistente en el cuerpo es 400). El detalle trae `asset` (id, código, nombre, estado y categoría) y `daysSinceRequest` (también en el listado).	G2, G6, G7
10	Técnicos completos: CRUD (sin borrar), filtros `?specialtyId=&isAvailable=`, `specialtyCount`, especialidades con `seniorityInMonths`, asociar (400/404/409) y desasociar (200 con `Resultado`, como el DELETE de clase; 404).	E1–E9, E16–E20
11	Especialidades completas: CRUD (sin borrar), nombre único (409), `technicianCount` y detalle con técnicos (nivel y disponibilidad).	E10–E15
12	`CatalogoService` ya no usa el DbContext: pasa por `ICatalogoRepository` (ubicaciones y tipos de mantenimiento).	A2
13	Repuestos: entidades `Repuesto` y `RepuestoProveedor` (tablas `SpareParts` y `SparePartSuppliers`, con CHECKs de stock ≥ 0, unidad, precio y días). CRUD sin borrar, `below-minimum`, `isBelowMinimum`, `shortage` (nunca negativo), `supplierCount`, asociar proveedor (201/404/409) y actualizar sus condiciones (200/404).	B2, F1, F3–F9, F16–F21
14	Proveedor: el detalle trae `spareParts` con precio y días; no se elimina si tiene repuestos.	F14
15	Datos semilla de TI / data center: entre 8 y 12 registros por tabla. Casos borde: repuestos bajo el mínimo (y uno justo en el mínimo), activos UnderMaintenance, OutOfService y Decommissioned, técnico con 3 especialidades, técnico sin especialidades, ubicación y proveedor inactivos, y un activo que ya requiere preventivo.	B5, B6
15b	`[FechaNoFutura]`, una DataAnnotation propia en `Proyecto.Domain/Validaciones`, se aplica a la fecha de certificación y a la fecha de adquisición. Así toda validación de forma pasa por ModelState y en los servicios solo quedan reglas que consultan la base de datos (Clase 4-2).	A7, A9
16	Se borró `database/MantenimientoActivos.sql`, se actualizó el README y se agregaron 11 pruebas (`PruebasReglasNegocio.cs`).	B1

Criterio general: donde la rúbrica y el patrón de clase coinciden se siguió el patrón de clase; donde la rúbrica pide algo específico (404 del activo en órdenes) manda la rúbrica.

Todos los conteos se hacen en SQL Server con un solo `GROUP BY` (`Contar...PorXAsync`) sin cargar las colecciones. Los campos de fecha (antigüedad y días) se calculan en Mapster con `MesesTranscurridos` y `DiasTranscurridos`. Las rutas y los nombres JSON siguen en inglés.

Pendiente en la demo (tú, en Visual Studio):

```
dotnet build Proyecto.slnx
dotnet ef migrations add AddSparePartsAndItSeedData --project Proyecto.Infraestructure --startup-project
Proyecto.WebAPI -- --environment Development
dotnet ef database drop --project Proyecto.Infraestructure --startup-project Proyecto.WebAPI -- --environment
Development
dotnet ef database update --project Proyecto.Infraestructure --startup-project Proyecto.WebAPI -- --environment
```

```
Development
dotnet test Proyecto.slnx
```

Si algo no compila o falla una prueba, pégame el error y lo corrijo en la demo antes de pasarlo al real.

---

2. Prompts para el repo real (en orden)

> Reglas que aplican a todos los prompts (ya vienen dentro de cada uno): no tocar `appsettings.Development.json`, no hacer commits, los DTOs son `class` (no `record`) y viven en `Proyecto.Domain/DTO`, y los comentarios deben explicar el porqué (nada genérico).

Prompt 0 — Traducción del repo al español (Andrey)

Commit: `refactor: translate code identifiers to spanish`

```
Trabaja en C:\Users\Andrey\source\repos\Proyecto (ASP.NET Core 10, EF Core 10, Clean Architecture).
Usa como referencia la carpeta C:\Users\Andrey\Downloads\ProyectoWeb-main\ProyectoWeb-main: es la
misma solución, ya traducida. Copia de ella los nombres exactos; no inventes otros.

Objetivo: traducir al español todos los identificadores de C# del repo, sin cambiar la base de datos
ni el contrato HTTP.

1. Renombra clases, archivos, propiedades, métodos, parámetros y variables al español, con los mismos
   nombres que la demo. Ejemplos: Asset→Activo, AssetCategory→CategoriaActivo, Location→Ubicacion,
   Supplier→Proveedor, MaintenanceOrder→OrdenMantenimiento, MaintenanceType→TipoMantenimiento,
   Technician→Tecnico, Specialty→Especialidad, TechnicianSpecialty→TecnicoEspecialidad,
   AppDbContext→ContextoAplicacion, Result<T>→Resultado<T>, BaseResult→ResultadoBase (carpeta
   DTO/PatternResult→DTO/ResultadoPatron), Success/Failure/Paged→Exito/Fallo/Paginado,
   MapsterConfig→ConfiguracionMapster, CatalogSeeder→InicializadorCatalogos, SeedIds→IdentificadoresSemilla,
   AssetService→ActivoService, IFileStorageService→IAlmacenamientoArchivosService,
   GetAllAsync→ObtenerTodosAsync, GetByIdAsync→ObtenerPorIdentificadorAsync, AddAsync→AgregarAsync,
   UpdateAsync→ActualizarAsync, DeleteAsync→EliminarAsync, Id→Identificador, _context→_contexto,
   _mapper→_mapeador, result→resultado. Los DTOs: AssetCreateDto→ActivoCrearDto,
AssetDetailDto→ActivoDetalleDto, etc.
2. Base de datos SIN cambios: tablas y columnas siguen en inglés. Cada propiedad renombrada debe
   tener HasColumnName("NombreOriginalEnIngles") en ContextoAplicacion (la demo lo agrupa al final de
   OnModelCreating). Las llaves, índices, CHECKs y FKs quedan con los mismos nombres.
3. Contrato HTTP SIN cambios: las rutas ([Route("api/Asset")], etc.) quedan en inglés y cada
   propiedad de los DTOs lleva [System.Text.Json.Serialization.JsonPropertyName("nombreEnIngles")]
   con el nombre camelCase que tenía antes (id, name, assetCode, categoryId, imageUrl...). Resultado:
   isSuccess, statusCode, title, detail, data, pageNumber, pageSize, totalRecords, totalPages.
   Los parámetros de ruta conservan su nombre público con [FromRoute(Name = "pageNumber")].
4. Las migraciones ya existentes no se renombran ni se editan a mano, salvo el typeof del DbContext en
   los .Designer.cs y el snapshot (AppDbContext → ContextoAplicacion, archivo
```

```
   ContextoAplicacionModelSnapshot.cs), que deben seguir compilando.
5. Mensajes al usuario en español, igual que antes. Comentarios en español explicando el porqué.

Al terminar: compila (dotnet build Proyecto.slnx). Después ejecuta
dotnet ef migrations add TraduccionEspanol --project Proyecto.Infraestructure --startup-project Proyecto.WebAPI
-- --environment Development
y muéstrame el método Up(). Debe quedar VACÍO: si tiene RenameColumn, DropColumn o AddColumn, falta
un HasColumnName. Corrígelo, borra esa migración y genérala de nuevo hasta que Up() esté vacío.
No toques appsettings.Development.json ni hagas commits.
```

Prompt 1 — Excepciones, manejador global, traceId y Serilog (Austin)

Commit: `feat: add business exceptions and global exception handler`

```
Trabaja en C:\Users\Andrey\source\repos\Proyecto. Referencia: la demo en
C:\Users\Andrey\Downloads\ProyectoWeb-main\ProyectoWeb-main (copia la misma solución).

1. Proyecto.Domain/Excepciones: ExcepcionNegocio (abstracta, con CodigoEstado y Titulo),
   ExcepcionSolicitudInvalida (400, "Solicitud incorrecta"), ExcepcionNoEncontrado (404, "No encontrado")
   y ExcepcionConflicto (409, "Conflicto"). Igual que la demo.
2. ResultadoBase: agrega MarcaTiempo (JSON "timestamp", DateTime.UtcNow) e IdentificadorTraza
   (JSON "traceId", se omite cuando es null).
3. Proyecto.WebAPI/Middleware/ManejadorGlobalExcepciones.cs implementando IExceptionHandler: si es
   ExcepcionNegocio responde su código con Resultado<object>.Fallo, LogWarning con ruta, código,
   mensaje y traceId; si no, LogError y 500 "Error interno" sin detalles internos. Siempre asigna
   IdentificadorTraza = Activity.Current?.Id ?? HttpContext.TraceIdentifier.
4. Program.cs: AddExceptionHandler<ManejadorGlobalExcepciones>(), AddProblemDetails(), y
   app.UseExceptionHandler() como primer middleware. En InvalidModelStateResponseFactory asigna
   también IdentificadorTraza.
5. Serilog: paquete Serilog.AspNetCore 10.0.0 en WebAPI; builder.Host.UseSerilog con
   ReadFrom.Configuration, Enrich.FromLogContext, WriteTo.Console y WriteTo.File("Logs/log-.txt",
   diario, 14 archivos), sin archivo cuando el entorno es "Testing". app.UseSerilogRequestLogging()
   después de UseExceptionHandler. En appsettings.json reemplaza "Logging" por "Serilog":
   MinimumLevel Default Information, Override Microsoft.AspNetCore y Microsoft.EntityFrameworkCore en Warning.
   Verifica que Logs/ esté en .gitignore.
6. En TODOS los servicios cambia cada `return Resultado<...>.Fallo(400|404|409, ..., mensaje);` por
   `throw new ExcepcionSolicitudInvalida|ExcepcionNoEncontrado|ExcepcionConflicto(mensaje);` con el mismo
   mensaje. En los controladores quita los `if (!resultado.EsExitoso)` que quedan sin sentido.

Compila y prueba en Scalar: GET /api/Asset/9999 debe dar 404 con traceId y timestamp, y un POST
inválido 400 con traceId. No toques appsettings.Development.json ni hagas commits.
```

Prompt 2 — Activos: filtros, campos calculados, resumen de órdenes y PATCH (Andrey)

Commit: `feat: add asset filters, computed fields and order summary`

```
Trabaja en C:\Users\Andrey\source\repos\Proyecto. Referencia: la demo en
C:\Users\Andrey\Downloads\ProyectoWeb-main\ProyectoWeb-main (ActivoRepository, ActivoService,
ConfiguracionMapster, ActivoListaDto, ActivoDetalleDto, CategoriaResumenActivoDto, OrdenResumenActivoDto).

1. ConfiguracionMapster: agrega los métodos estáticos MesesTranscurridos(DateTime) (meses completos,
   nunca negativo) y DiasTranscurridos(DateTime) (días, nunca negativo).
2. Repositorio: método privado Filtrar(categoriaId, estado) usado por ObtenerTodosAsync,
   ObtenerPaginacionAsync y ContarAsync (los tres con parámetros opcionales). Agrega
   ContarOrdenesPorActivoAsync(IEnumerable<int>) que devuelve Dictionary<int,int> con un solo
   GroupBy en SQL (sin Include). En ObtenerPorIdentificadorAsync incluye
   OrdenesMantenimiento.OrderByDescending(SolicitadoEn).ThenInclude(TipoMantenimiento).
3. DTOs: ActivoListaDto suma categoryId, ageInMonths y maintenanceOrderCount. ActivoDetalleDto suma
   category {id, name, preventiveThresholdHours}, ageInMonths, hoursSinceLastMaintenance,
   preventiveMaintenanceDue (horas desde el último mantenimiento >= umbral de la categoría),
   maintenanceOrderCount y maintenanceOrders [{id, workOrderCode, requestedAt, maintenanceTypeName,
   priority, status}].
4. Servicio: MapearListaConConteoAsync y MapearDetalleAsync completan CantidadOrdenes con el
   diccionario. En Mapster ignora CantidadOrdenes porque lo asigna el servicio.
5. Controlador: GET /api/Asset y la paginación reciben
   [FromQuery(Name="categoryId")] [Range(1, int.MaxValue, ErrorMessage=...)] int? y
   [FromQuery(Name="status")]
[RegularExpression("^(Operational|UnderMaintenance|OutOfService|Decommissioned)$", ErrorMessage=...)] string?.
   La validación de forma es DataAnnotation (ModelState → 400), no un if en el servicio.
   En el repositorio, sin filtro status se excluyen los Decommissioned (como Status == true en clase).
   ObtenerParaActualizarAsync filtra Estado != "Decommissioned", igual que GetForUpdateAsync de clase:
   PUT, PATCH, DELETE e imagen sobre un activo dado de baja responden 404 "no existe o está dado de baja".
   El GET por id sí lo devuelve (su estado e historial se siguen consultando). Quita los if de
   "está dado de baja" → 409, que quedan sin uso.
6. ActualizarEstadoActivoDto acepta Operational|OutOfService|Decommissioned. En ActualizarEstadoAsync,
   si el estado cambia y TieneOrdenesAbiertasAsync es true → ExcepcionConflicto (409).

Compila y prueba: GET /api/Asset?status=OutOfService, GET /api/Asset/1 (resumen y conteo iguales) y
PATCH /api/Asset/{id}/status sobre un activo con orden abierta (409). No hagas commits.
```

Prompt 3 — Categorías y órdenes (Austin)

Commit: `feat: add category asset summary and order asset details`

```
Trabaja en C:\Users\Andrey\source\repos\Proyecto. Referencia: la demo en
C:\Users\Andrey\Downloads\ProyectoWeb-main\ProyectoWeb-main.

Categorías:
1. Repositorio: ContarActivosPorCategoriaAsync(IEnumerable<int>) con un GroupBy en SQL, y
   ObtenerPorIdentificadorAsync con Include(Activos.OrderBy(CodigoActivo)).
2. DTOs: CategoriaActivoListaDto y CategoriaActivoDetalleDto suman assetCount; el detalle suma
   assets [{id, assetCode, name, status}] (ActivoResumenCategoriaDto). Agrega el
   JsonPropertyName("codePrefix") que falta en PrefijoCodigo.
```

```
3. Servicio: el listado, la paginación y el detalle completan CantidadActivos con el diccionario;
   ActualizarAsync devuelve el detalle consultado de nuevo.

Órdenes:
4. AgregarAsync: si el activo no existe → ExcepcionNoEncontrado (404, lo pide la rúbrica). Si el tipo
   o el proveedor no existen → ExcepcionSolicitudInvalida (400, patrón de clase: Id inexistente en el cuerpo).
5. ObtenerPorIdentificadorAsync: Include(Activo).ThenInclude(Categoria).
6. DTOs: OrdenMantenimientoDetalleDto suma asset {id, assetCode, name, status, categoryId,
   categoryName} (ActivoResumenOrdenDto). El listado y el detalle suman daysSinceRequest
   (DiasTranscurridos en Mapster).

Compila y prueba en Scalar GET /api/AssetCategory/1, GET /api/MaintenanceOrder/1 y un POST de orden
con assetId inexistente (404). No hagas commits.
```

Prompt 4 — Técnicos, especialidades y catálogos (Andrey)

Commit: `feat: add technician and specialty management`

```
Trabaja en C:\Users\Andrey\source\repos\Proyecto. Referencia: la demo en
C:\Users\Andrey\Downloads\ProyectoWeb-main\ProyectoWeb-main. Copia TecnicoRepository,
EspecialidadRepository, CatalogoRepository (con sus interfaces), TecnicoService, EspecialidadService,
CatalogoService, los DTOs Tecnico*, Especialidad*, SolicitudEspecialidadTecnicoDto,
UbicacionListaDto y TipoMantenimientoListaDto, los mapeos de Mapster y los controladores
TecnicosController, EspecialidadesController y CatalogosController. Adáptalos si algo difiere.

Endpoints (rutas en inglés):
- GET /api/technicians?specialtyId=&isAvailable=, GET/PUT /api/technicians/{id}, POST /api/technicians
  (identificación única → 409).
- POST /api/technicians/{id}/specialties/{specialtyId} body {certificationLevel, certifiedSince}:
  201; 400 si el nivel no es Basic/Intermediate/Advanced o la fecha es futura; 404 si no existe el
  técnico o la especialidad; 409 si ya la tiene. DELETE de la misma ruta: 200 con Resultado del técnico
  actualizado (en clase, 204 solo se usa cuando la API no devuelve Resultado<T>) o 404.
- La fecha futura se valida con una DataAnnotation propia: crea Proyecto.Domain/Validaciones/
  FechaNoFuturaAttribute.cs (igual que la demo) y aplícala a certifiedSince y a la fecha de adquisición
  de ActivoCrearDto y ActivoActualizarDto. El filtro specialtyId lleva [Range(1, int.MaxValue)] en el
  parámetro. Ninguna validación de forma va como if en el servicio.
- GET/POST /api/specialties, GET/PUT /api/specialties/{id} (nombre único → 409).
- GET /api/locations (solo activas) y GET /api/maintenance-types.
Campos calculados: specialtyCount y technicianCount con GroupBy en SQL (Contar...PorXAsync), y
seniorityInMonths por especialidad (MesesTranscurridos de CertificadoDesde). El detalle del técnico
trae specialties [{id, name, certificationLevel, certifiedSince, seniorityInMonths}] y el de la
especialidad, technicians [{id, fullName, certificationLevel, certifiedSince, isAvailable}].
Regla de arquitectura: ningún archivo de Proyecto.Application puede usar el DbContext ni
Microsoft.EntityFrameworkCore. DTOs como class con DataAnnotations y mensajes en español.
Registra todo en Program.cs.
```

```
Compila y prueba en Scalar asociar, duplicar (409), fecha futura (400) y quitar una especialidad (200).
No hagas commits.
```

Prompt 5 — Repuestos y proveedor con repuestos (Austin)

Commit: `feat: add spare parts with supplier pricing`

```
Trabaja en C:\Users\Andrey\source\repos\Proyecto. Referencia: la demo en
C:\Users\Andrey\Downloads\ProyectoWeb-main\ProyectoWeb-main (Repuesto, RepuestoProveedor,
ContextoAplicacion, RepuestoRepository, RepuestoService, RepuestoController y los DTOs Repuesto*,
RepuestoProveedorDto, ProveedorRepuestoDto y CondicionesProveedorRepuestoDto).

1. Entidades Repuesto (Codigo, Nombre, Descripcion, UnidadMedida, StockActual, StockMinimo, CreadoEn)
   y RepuestoProveedor (llave compuesta RepuestoId + ProveedorId, PrecioOfrecido, DiasEntregaEstimados).
   Proveedor gana la colección RepuestosProveedor.
2. ContextoAplicacion: tablas SpareParts y SparePartSuppliers con columnas en inglés (Id, Code, Name,
   Description, UnitOfMeasure, CurrentStock, MinimumStock, CreatedAt; SparePartId, SupplierId,
   OfferedPrice decimal(18,2), EstimatedLeadTimeDays), índice único en Code, FKs Restrict y los CHECKs
   CK_SpareParts_Stock, CK_SpareParts_UnitOfMeasure (Unit, Box, Pack, Meter, Roll, Kit),
   CK_SparePartSuppliers_OfferedPrice y CK_SparePartSuppliers_LeadTimeDays.
3. Endpoints: GET/POST /api/SparePart, GET /api/SparePart/below-minimum, paginación,
   GET/PUT /api/SparePart/{id} (código único → 409; stock negativo → 400),
   POST /api/SparePart/{id}/suppliers/{supplierId} (201; 404 si falta el repuesto o el proveedor;
   409 si ya está asociado o el proveedor está inactivo) y PUT de la misma ruta (200 o 404).
4. Calculados: isBelowMinimum (StockActual < StockMinimo, la misma condición que below-minimum),
   shortage = Math.Max(StockMinimo - StockActual, 0) y supplierCount con GroupBy en SQL. El detalle
   trae suppliers [{id, name, offeredPrice, estimatedLeadTimeDays}].
5. Proveedor: el detalle trae spareParts [{id, code, name, offeredPrice, estimatedLeadTimeDays}]
   (Include en el repositorio); no se puede eliminar si tiene repuestos; ActualizarAsync devuelve el
   detalle consultado de nuevo. Ignora RepuestosProveedor en los mapeos de entrada.
6. Migración: dotnet ef migrations add AddSpareParts --project Proyecto.Infraestructure
   --startup-project Proyecto.WebAPI -- --environment Development. Revisa que solo cree las dos tablas.

Compila, aplica la migración y prueba en Scalar. No hagas commits.
```

Prompt 6 — Datos semilla de TI (Andrey)

Commit: `feat: seed it data center catalog with edge cases`

```
Trabaja en C:\Users\Andrey\source\repos\Proyecto. Reemplaza IdentificadoresSemilla.cs e
InicializadorCatalogos.cs por los de la demo
(C:\Users\Andrey\Downloads\ProyectoWeb-main\ProyectoWeb-main\Proyecto.Infraestructure\Data\Seed),
adaptando nombres si alguno difiere. Contexto: equipos de TI / data center. Incluye categorías,
ubicaciones, proveedores, especialidades, técnicos, técnico-especialidad, tipos de mantenimiento,
activos, órdenes, repuestos y repuesto-proveedor, con 8 a 12 registros por tabla y fechas fijas.

Comprueba antes de generar la migración que se cumplan los CHECK: prefijos de 2 a 5 letras
```

```
mayúsculas, códigos de activo PREFIJO-0001, horas al último mantenimiento <= horas de uso,
ClosedAt >= StartedAt, stock >= 0. Revisa también que los estados sean coherentes: un activo
UnderMaintenance tiene una orden abierta, y los OutOfService y Decommissioned solo tienen órdenes cerradas.
Casos borde: repuestos bajo el mínimo y uno justo en el mínimo; activos UnderMaintenance,
OutOfService y Decommissioned; técnico con 3 especialidades; ubicación y proveedor inactivos.

Luego:
dotnet ef migrations add SeedItCatalog --project Proyecto.Infraestructure --startup-project Proyecto.WebAPI --
--environment Development
dotnet ef database drop -f --project Proyecto.Infraestructure --startup-project Proyecto.WebAPI -- --
environment Development
dotnet ef database update --project Proyecto.Infraestructure --startup-project Proyecto.WebAPI -- --environment
Development
Si el update falla por una restricción, corrige el dato semilla, borra la migración y genérala de nuevo.
No hagas commits.
```

Prompt 7 — Documentación Scalar y logging por endpoint (Austin)

Commit: `docs: document endpoints and add request logging`

```
Trabaja en C:\Users\Andrey\source\repos\Proyecto. Referencia: los controladores de la demo en
C:\Users\Andrey\Downloads\ProyectoWeb-main\ProyectoWeb-main\Proyecto.WebAPI\Controllers.

1. En Proyecto.WebAPI.csproj: <GenerateDocumentationFile>true</GenerateDocumentationFile> y
   <NoWarn>$(NoWarn);1591</NoWarn>.
2. En CADA controlador: inyecta ILogger<T> (campo _registro); en cada acción, LogInformation al iniciar
   (con los parámetros importantes como propiedades estructuradas {Id}, {Pagina}...) y al completar.
   Los Warning de las fallas los registra el ManejadorGlobalExcepciones: no atrapes excepciones en el
controlador.
3. En CADA acción: /// <summary> que diga qué hace en una línea, <remarks> cuando haya reglas
   (409 por órdenes abiertas, etc.) y [ProducesResponseType(typeof(Resultado<TipoDto>), 200|201)] más
   [ProducesResponseType(typeof(Resultado<object>), 400|404|409)] según lo que realmente pueda pasar.
   [Produces("application/json")] en la clase. Los DELETE que responden 204 llevan
   [ProducesResponseType(StatusCodes.Status204NoContent)].
4. Los POST de creación responden con CreatedAtRoute usando el Name del GET por id.

Compila, abre /scalar y confirma que cada endpoint muestra el resumen, los códigos y los esquemas.
No hagas commits.
```

Prompt 8 — Pruebas, README y limpieza (Andrey)

Commit: `test: add api tests and update readme`

```
Trabaja en C:\Users\Andrey\source\repos\Proyecto. Referencia: la demo en
C:\Users\Andrey\Downloads\ProyectoWeb-main\ProyectoWeb-main.

1. Copia el proyecto Proyecto.WebAPI.Tests (FabricaApi, PruebasEndpointsTecnico,
```

```
   PruebasContratoActivo, PruebasReglasNegocio) y agrégalo a Proyecto.slnx. Usa InMemory y el
   entorno "Testing"; ajusta nombres si alguno difiere.
2. Borra database/MantenimientoActivos.sql (ya no se usa: la base la crean las migraciones).
3. Reemplaza el README por el de la demo, revisando que las rutas coincidan con los controladores.
4. Ejecuta dotnet test Proyecto.slnx y corrige lo que falle en el código, no en las pruebas (salvo
   que la prueba esté mal; en ese caso explícame por qué).
No hagas commits.
```

Prompt 9 — Verificación final contra la rúbrica (cualquiera de los dos, sin commit)

```
Trabaja en C:\Users\Andrey\source\repos\Proyecto. No modifiques código: solo verifica.
1. dotnet ef database drop -f y dotnet ef database update (con -- --environment Development) sobre una
   base vacía; confirma que todas las migraciones aplican y que cada tabla principal tiene entre 8 y 12 filas.
2. dotnet build y dotnet test sin errores.
3. Recorre la rúbrica del Avance 1 (A1–A9, B1–B6, C1–C20, D1–D11, E1–E20, F1–F21, G1–G11) y, para cada
   rubro, dime el endpoint o archivo que lo cumple o qué falta. Confirma también que
   Proyecto.Application no referencia el DbContext y que no queda ningún return ...Fallo(400/404/409)
   en los servicios.
Entrégame la lista lista para pasarla a la tabla de verificación.
```

---

3. Después de los prompts

1. Subir la rama y hacer el merge a `main` (como en los avances anteriores).
2. Actualizar la tabla de verificación con el hash del commit final (puedo regenerarla).
3. Guion del video de 15 minutos, en este orden: arquitectura y Scalar → activos (filtros, detalle, PATCH 409) → categorías → técnicos y especialidades → repuestos y proveedores → órdenes → errores con traceId y log.